



死锁

Deadlocks



主要内容

- 死锁模型的定义 System Model
- 死锁产生的必要条件 Deadlock Characterization
- 解决死锁的方法 Methods for Handling Deadlocks
- 死锁的检测 Deadlock Detection
- 死锁的恢复 Recovery from Deadlock
- 死锁的避免 Deadlock Avoidance
- 死锁的预防 Deadlock Prevention

资源的使用

■ 在一般操作模式下，进程只能按如下顺序使用资源：

■ ①申请：如果申请不能立即被允许，那么申请进程必须等待，直到它获得该资源为止。

■ ②使用：进程对资源进行操作。

■ ③释放：进程释放资源

```
semaphore s
void process() {
    P(s);
    use_resource();
    V(s);
}
```

■ 资源的申请与释放为系统调用。可以通过信号量的P与V操作或通过互斥锁的获取与释放来完成。

■ 对于进程和线程的每次使用，操作系统会检查以确保使用进程已经申请并获得了资源。

■ 系统表记录了每个资源是否空闲或已被分配，分配给了哪个进程。如果进程正在申请的资源正在为其他进程所使用，那么该进程会增加到该资源的等待队列。



使用资源引发死锁

- 对于多个进程使用多个资源的场景，有可能由于申请顺序不当引发死锁。
- 左图：无死锁风险
- 右图：存在潜在死锁风险

```
semaphore s1=1, s2=1;
void process_A() {
    P(s1);
    P(s2);
    use_both_resources();
    V(s1);
    V(s2);
}
void process_B() {
    P(s1);
    P(s2);
    use_both_resources();
    V(s1);
    V(s2);
}
```

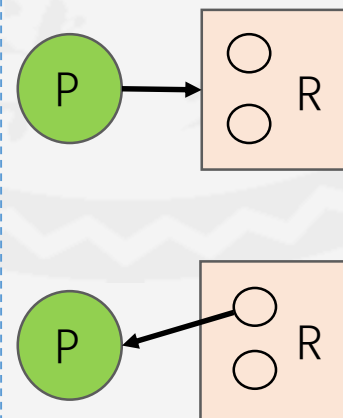
```
semaphore s1=1, s2=1;
void process_A() {
    P(s1);
    P(s2);
    use_both_resources();
    V(s1);
    V(s2);
}
void process_B() {
    P(s2);
    P(s1);
    use_both_resources();
    V(s1);
    V(s2);
}
```

死锁的定义和发生的必要条件

- 死锁可以正式定义如下：存在一组等待进程 $\{P_0, P_1, \dots, P_n\}$ ，如果集合中的每个进程都在等待只有集合中另一个进程才能引发的事件，则该集合中的进程处于死锁状态。
- 如果四个必要条件同时成立，则可能会出现死锁。
 - **互斥 (Mutual exclusion)**：至少一个资源为**临界资源 (critical resource)**，一次只能有一个进程使用
 - **持有并等待 (Hold and wait)**：一个进程至少持有一个资源，并等待另一个资源，而该资源为其他进程所占
有
 - **不可抢占 (No preemption)**：资源只能由持有该资源的进程完成任务后自愿释放
 - **循环等待 (Circular wait)**： P_0 正在等待由 P_1 持有的资源，而 P_1 正在等待由 P_2 持有的资源， $\dots$ ， P_{n-1} 正在等待由 P_n 持有的资源，而 P_n 正在等待由 P_0 持有的资源。

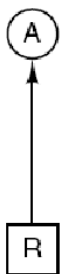
资源分配图 (Resource-Allocation Graph)

- 资源分配图由一个节点集合 V (a set of vertices V) 和一个边集合 E (a set of edges E) 组成。
- 节点 V 分成二种类型： $P = \{P_1, P_2, \dots, P_n\}$ 代表进程（圆形）， $R = \{R_1, R_2, \dots, R_m\}$ 代表资源（矩形）。一种资源可能有多个实例（instances）（矩形内的圆点）
 - 例如：右图中的 R 有2个实例。比如，一台计算机上挂接了两台一样的打印机。
- 边 E 分为二种类型：
 - 申请边（request edge），由进程 P 指向资源 R ，表示进程 P 申请得到资源 R 的一个实例
 - 分配边（assignment edge，由资源 R 指向进程 P ，表示资源 R 的一个实例已分配给进程 P



资源分配图 (Resource-Allocation Graph)

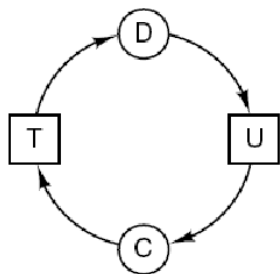
- 如果图中不含环路，无死锁
- 如果图包含一个环路，那么：
 - 如果每个资源类型只有一个实例，则发生死锁
 - 如果每个资源类型有多个实例，那么就有可能出现死锁



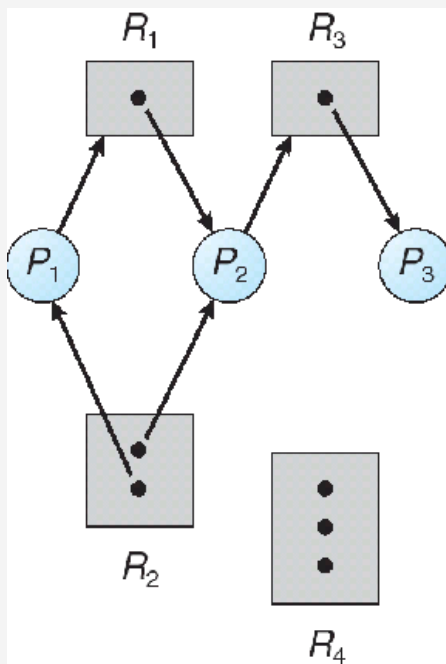
(a)



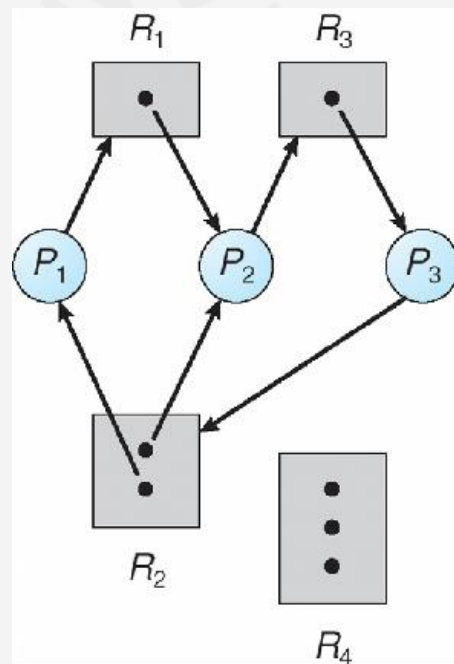
(b)



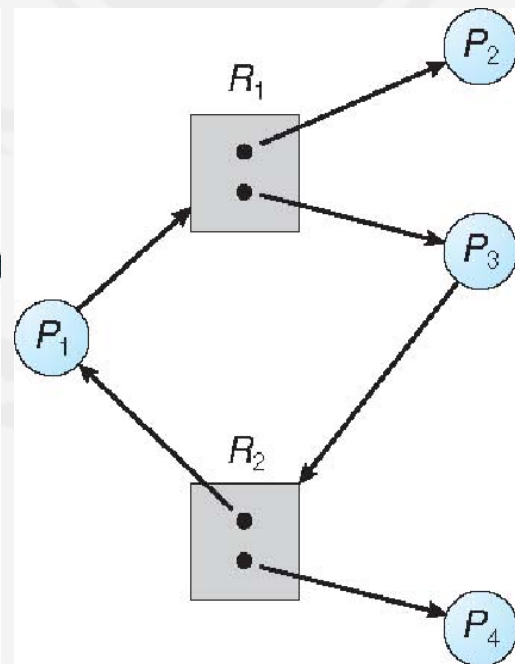
(c)



没有环没有死锁



存在环并死锁



存在环并未死锁



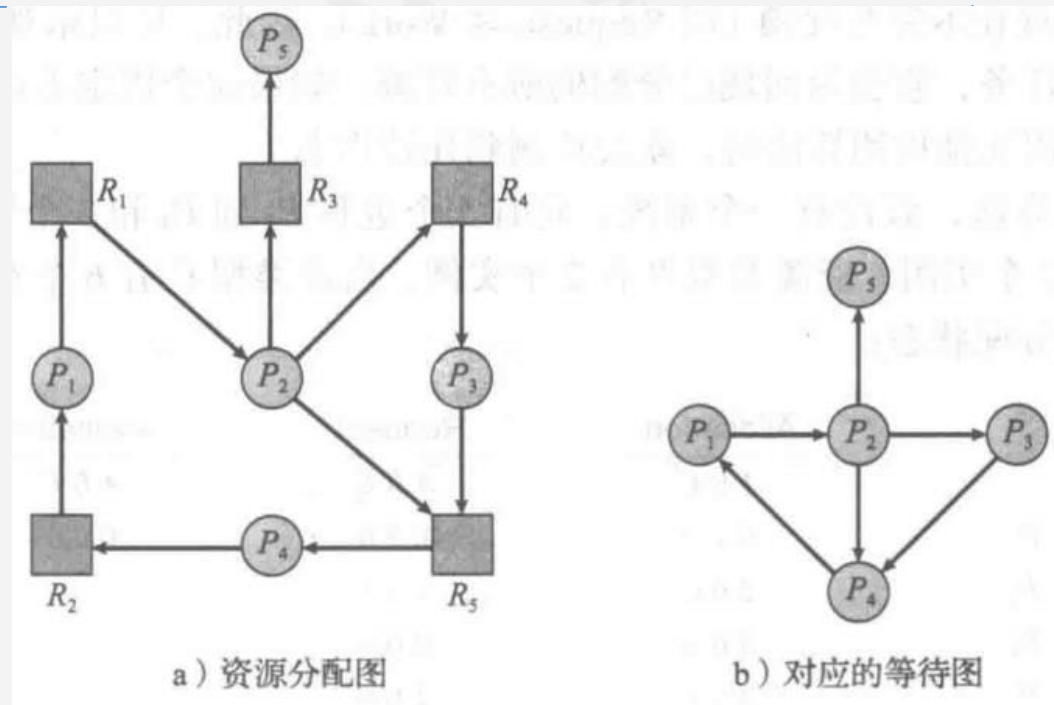
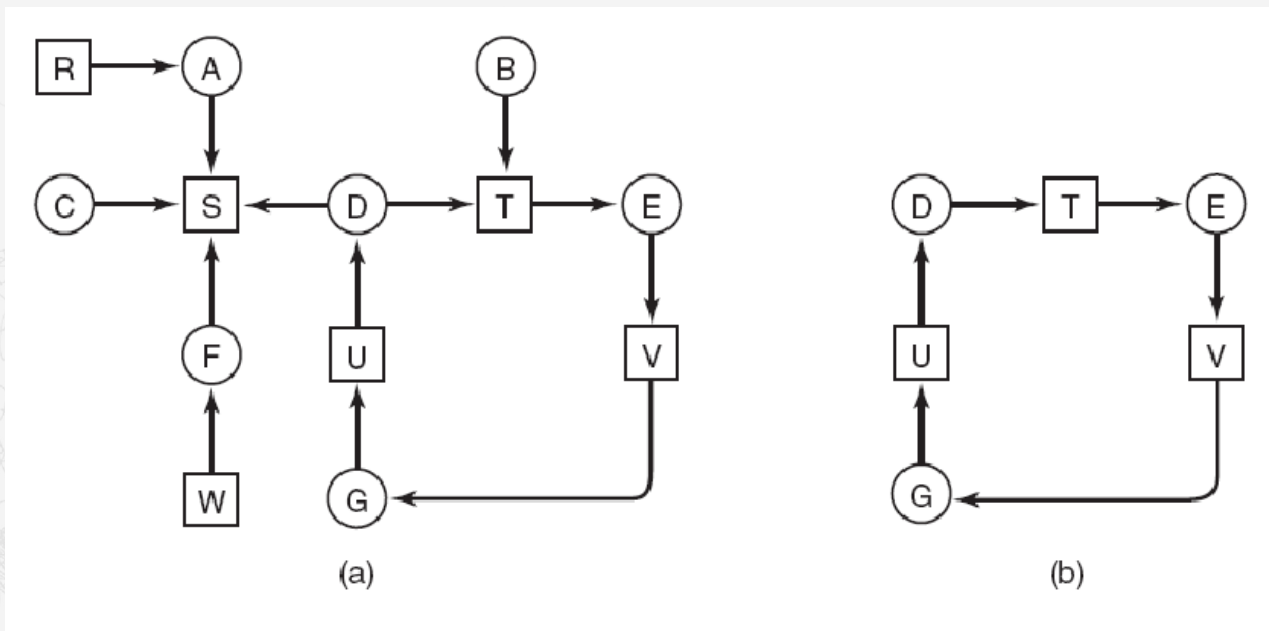
死锁处理方法

- 处理死锁问题的方法：
 - 可以忽视这个问题，认为死锁不可能在系统内发生（**鸵鸟策略**）。主流操作系统广泛使用。
 - **死锁检测与恢复 (deadlock detection and recovery)**：可以允许系统进入死锁状态，然后检测它，并加以恢复。
 - **死锁避免 (deadlock avoidance)**：小心分配资源，确保系统不会进入死锁状态。
 - **死锁预防 (deadlock prevention)**：破坏死锁的四个必要条件，使其至少一个不成立。

单实例资源的死锁检测 (deadlock detection)

■ **等待 (wait-for) 图**: 资源分配图中删除所有资源节点, 并合并部分边, 即得到等待图。

■ P_i 到 P_j 的边, 代表 P_i 等待 P_j 释放一个 P_i 所需的资源。



多实例资源的死锁检测 (deadlock detection)

■ 资源分配矩阵 (Resources Allocation Matrix) : n 个进程使用 m 种资源

(E) Existence: m 种资源实例总量

(A) Available: 长度为 m 的向量, 表示各种资源的可用实例数量

(C) Allocation: $n \times m$ 矩阵, 表示每个进程的每种资源的当前分配数量。

(R) Request: $n \times m$ 矩阵, 表示当前每个进程的每种资源的当前请求。

(M) Max: $n \times m$ 矩阵, 表示每个进程的对每种资源的最大需求总量。

$$E_j = \sum_{i=1}^n C_{ij} + A_j$$

$$M_{ij} = C_{ij} + R_{ij}$$

Resources in existence
($E_1, E_2, E_3, \dots, E_m$)

Resources available
($A_1, A_2, A_3, \dots, A_m$)

Current allocation matrix

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} & \cdots & C_{1m} \\ C_{21} & C_{22} & C_{23} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & C_{n3} & \cdots & C_{nm} \end{bmatrix}$$

Row n is current allocation to process n

Request matrix

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \cdots & R_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \cdots & R_{nm} \end{bmatrix}$$

Row 2 is what process 2 needs

多实例资源的死锁检测 (deadlock detection)

■ 例：3个进程（P1, P2, P3），使用4种资源，是否死锁？

■ 将A=(2, 1, 0, 0)分配给P3，P3顺利完成，归还本占有的(0, 1, 2, 0)，A变为(2, 2, 2, 0)

■ 将A=(2, 2, 2, 0)分配给P2，P2顺利完成，归还本占有的(2, 0, 0, 1)，A变为(4, 2, 2, 1)

■ 将A=(4, 2, 2, 1)分配给P1，P1顺利完成，归还本占有的资源。

$$E = \begin{pmatrix} 4 & 2 & 3 & 1 \end{pmatrix}$$

Tape drives
Plotters
Scanners
CD Roms

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

Tape drives
Plotters
Scanners
CD Roms

Current allocation matrix

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Request matrix

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

■ 由于存在 P3→P2→P1 这个执行序列，使得全部进程最终得到满足，没有发生死锁。我们把这个执行序列称为**安全序列**，存在安全序列（可能不唯一）的状态被称为**安全状态**，反之不存在安全序列的状态为**不安全状态**。



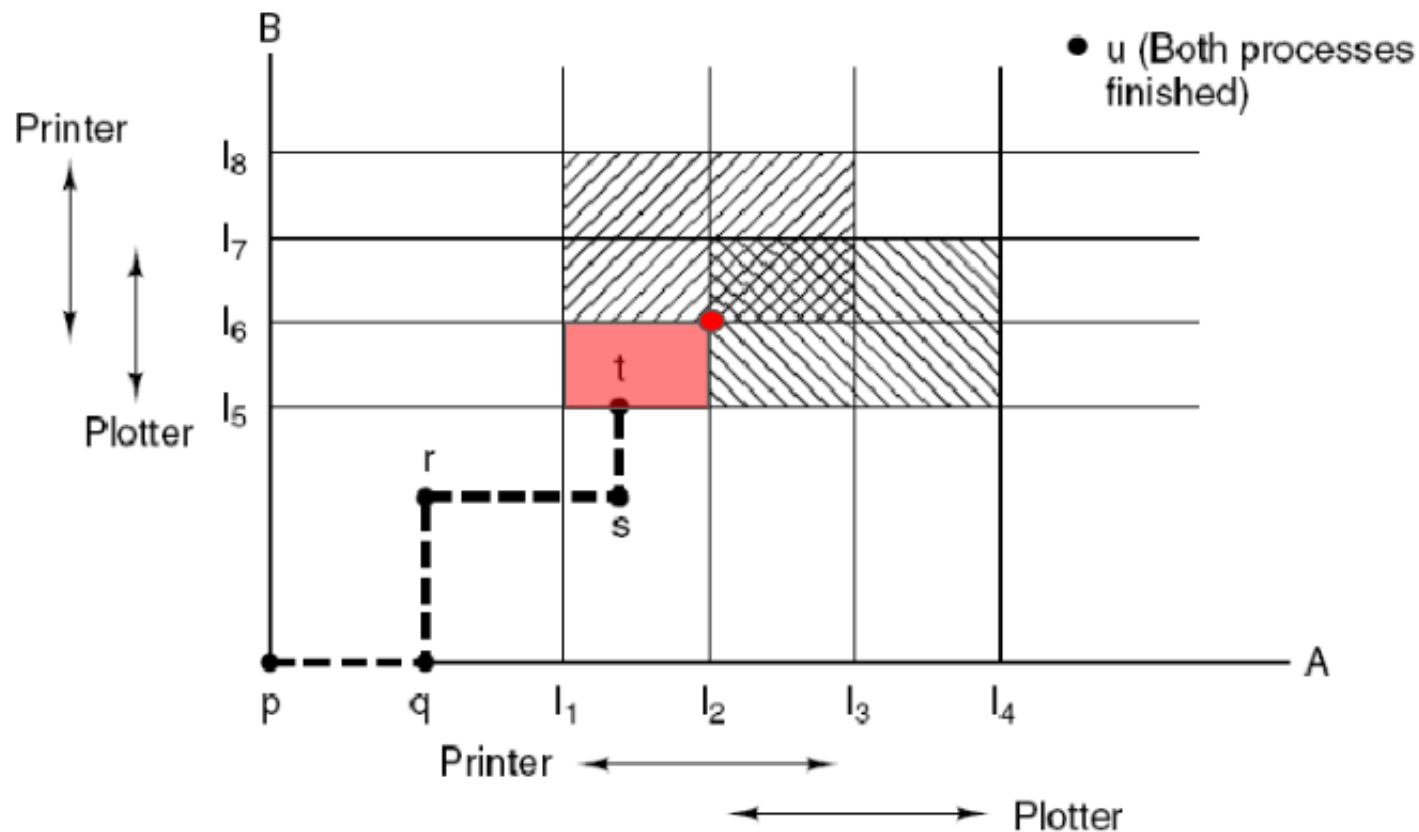
死锁的恢复 (deadlock recovery)

- 打破死锁有两种方法：
 - (1) 简单地终止一个或多个进程以打破循环等待。——进程终止
 - (2) 从一个或多个死锁进程那里抢占一个或多个资源。——资源抢占
- 进程终止有以下两种方法： (1) 终止所有死锁进程。 (2) 一次只终止一个进程直到取消死锁循环为止。(选择被终止的进程的条件是什么? 比如: 最年轻的进程)
- 如果选择资源抢占, 那么将必须考虑三个问题:
 - (1) 选择一个牺牲品 (2) 回滚 (3) 饥饿 (在代价因素中加上回滚次数, 回滚的越多则越不可能继续被作为牺牲品)

死锁的避免 (deadlock avoidance)

资源轨迹图 (resource trajectory)

- 当进程A运行时，轨迹向右，当进程B运行时，轨迹向上。只有这两种可能。最终的目的地是u点，代表A和B均运行结束。
- 在t点时，如果进程B运行，则轨迹向上，进入红色区域，最终必然到达右上角红点处，进程A、B死锁。
- 只适用于二个进程，并且需要获得以后如何申请资源的附加信息。





死锁的避免：银行家算法（Banker's Algorithm）

- 银行家算法：当进程申请一组资源时，如果剩余资源数量够分配的话，系统先试探的分配，但要确定这些资源的分配是否仍会使系统处于安全状态，如果是，就分配资源；否则，进程必须等待直到某个其他进程释放足够资源为止。
- 当新进程进入系统时，它必须说明其可能需要的各种类型资源实例的最大数量，这一数量不能超过当前系统资源的总和。

银行家算法 (Banker's Algorithm)

■ 单类资源的银行家算法。银行家共有10个资源（比如10万元），ABC已分配资源(Has)和最大需求资源(Max)如状态(a)所示，银行家剩余资源记录为Free。

■ 上图中B先申请2个资源，尝试分配后仍然为安全状态，故可以分配。

■ 下图中A申请了1个资源，尝试分配后进入不安全状态，故不可分配。

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3
(a)

	Has	Max
A	3	9
B	4	4
C	2	7

Free: 1
(b)

	Has	Max
A	3	9
B	0	—
C	2	7

Free: 5
(c)

	Has	Max
A	3	9
B	0	—
C	7	7

Free: 0
(d)

	Has	Max
A	3	9
B	0	—
C	0	—

Free: 7
(e)

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3
(a)

	Has	Max
A	4	9
B	2	4
C	2	7

Free: 2
(b)

	Has	Max
A	4	9
B	4	4
C	2	7

Free: 0
(c)

	Has	Max
A	4	9
B	—	—
C	2	7

Free: 4
(d)

银行家算法 (Banker's Algorithm)

- 多资源的银行家算法。
- 银行家所有的资源为E
- 进程已分配资源 (assigned), 按列求和为P
- 进程仍需要的资源 (needed)
- 银行家剩余资源为A。

	Process	Tape drives	Plotters	Printers	CD ROMs
A	3	0	1	1	
B	0	1	0	0	
C	1	1	1	0	
D	1	1	0	1	
E	0	0	0	0	

Resources assigned

	Process	Tape drives	Plotters	Printers	CD ROMs
A	1	1	0	0	
B	0	1	1	2	
C	3	1	0	0	
D	0	0	1	0	
E	2	1	1	0	

Resources still needed

E = (6342)
P = (5322)
A = (1020)

银行家算法 (Banker's Algorithm)

例题：银行家算法

有5个进程 P0-P4，使用3种资源A、B、C。各进程已分配资源为 **C**urrent-allocation，所需资源最大数量为 **M**ax-need，资源可用数量 **A**vailable。问：

- (1) 当前时刻是否是安全状态？
- (2) 如果当前安全，P4 请求 (2, 1, 0)，是否分配？
- (3) 在(2)基础上，P0 请求 (1, 0, 2)，是否分配？
- (4) 在(3)基础上，P1 请求 (1, 0, 2)，是否分配？

Available		
A	B	C
3	2	2

	Current allocation			Max-need		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	1	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

银行家算法 (Banker's Algorithm)

- (1) 当前时刻是否是安全状态?
- 题目中只给最大 Max-need 时, 要先求出还需资源数量 Request, 然后求安全序列。

Process	Current allocation			Max needed			Request		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	1	0	3	2	2	1	1	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

$$R_{ij} = M_{ij} - C_{ij}$$

$$E_j = \sum_{i=1}^n C_{ij} + A_j$$

Existence		
A	B	C
10	5	7

Available		
A	B	C
3	2	2

银行家算法 (Banker's Algorithm)

- $A = (3, 2, 2)$ ，此时P1和P3的申请可以满足，任选P1分配，P1执行完成后归还 $(2, 1, 0)$ ，A变为 $(5, 3, 2)$
- 安全序列：P1

Process	Current allocation			Max needed			Request			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	2	2
P1	2	1	0	3	2	2	1	1	2			
P2	3	0	2	9	0	2	6	0	0			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

银行家算法 (Banker's Algorithm)

- $A = (5, 3, 2)$ ，此时P3和P4的申请可以满足，任选P3分配，P3执行完成后归还 $(2, 1, 1)$ ，A变为 $(7, 4, 3)$
- 安全序列：P1→P3

Process	Current allocation			Max needed			Request			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	5	3	2
P1				3	2	2						
P2	3	0	2	9	0	2	6	0	0			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

银行家算法 (Banker's Algorithm)

- $A=(7, 4, 3)$ ，此时P0、P2和P4的申请可以满足，任选P0分配，P0执行完成后归还(0, 1, 0)，A变为(7, 5, 3)
- 安全序列：P1→P3→P0

Process	Current allocation			Max needed			Request			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	7	4	3
P1				3	2	2						
P2	3	0	2	9	0	2	6	0	0			
P3				2	2	2						
P4	0	0	2	4	3	3	4	3	1			

银行家算法 (Banker's Algorithm)

- $A = (7, 5, 3)$ ，此时P2和P4的申请可以满足，任选P2分配，P2执行完成后归还 $(3, 0, 2)$ ，A变为 $(10, 5, 5)$
- 安全序列：P1→P3→P0→P2

Process	Current allocation			Max needed			Request			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0				7	5	3				7	5	3
P1				3	2	2						
P2	3	0	2	9	0	2	6	0	0			
P3				2	2	2						
P4	0	0	2	4	3	3	4	3	1			

银行家算法 (Banker's Algorithm)

- $A=(10, 5, 3)$ ，此时P4的申请可以满足，P4分配，P4执行完成后归还 $(0, 0, 2)$ ，A变为 $(10, 5, 7)$
- 安全序列：P1→P3→P0→P2→P4，所有进程执行完成，说明(1)中T0时刻是安全状态

Process	Current allocation			Max needed			Request			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0				7	5	3				10	5	5
P1				3	2	2						
P2				9	0	2						
P3				2	2	2						
P4	0	0	2	4	3	3	4	3	1			

银行家算法 (Banker's Algorithm)

■ (2) 如果当前安全，P4 请求 (2, 1, 0)，是否分配？

■ 解：当前安全，P4请求 (2, 1, 0) 不超过A的 (3, 2, 2)，可尝试分配后得安全序列：P1→P3→P4→P0→P2，能分配

Process	Current allocation			Max needed			Request			Existence		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	10	5	7
P1	2	1	0	3	2	2	1	1	2			
P2	3	0	2	9	0	2	6	0	0			
P3	2	1	1	2	2	2	0	1	1			
P4	0→2	0→1	2→2	4	3	3	4→2	3→2	1→1			

Available		
A	B	C
3- >1	2- >1	2- >2

银行家算法 (Banker's Algorithm)

- 在(2)基础上, P0请求(1, 0, 2), 是否分配? 注意(2)的结论是可分配, 所以在(2)的基础上就是(2)分配了
- 解: P0 请求的(1, 0, 2)不超过A的(3, 2, 2), 可尝试分配后求安全序列, 结论是不存在安全序列, 不能分配。

Process	Current allocation			Max needed			Request			Existence		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0→1	1→1	0→2	7	5	3	7→6	4→4	3→1	10	5	7
P1	2	1	0	3	2	2	1	1	2			
P2	3	0	2	9	0	2	6	0	0			
P3	2	1	1	2	2	2	0	1	1			
P4	2	1	2	4	3	3	2	2	1			

Available		
A	B	C
1- >0	1- >1	2- >0

银行家算法 (Banker's Algorithm)

- 在(3)基础上, P1 请求 (1, 0, 2), 是否分配 ? 注意(3)的结论是不能分配, 所以在(3)的基础上就是(3)不分配

Process	Current allocation			Max needed			Request			Existence
	A	B	C	A	B	C	A	B	C	
P0	0	1	0	7	5	3	7	4	3	10 5 7
P1	2→3	1→1	0→2	3	2	2	1→0	1→1	2→0	
P2	3	0	2	9	0	2	6	0	0	Available A B C 1- >0 1- >1 2- >0
P3	2	1	1	2	2	2	0	1	1	
P4	2	1	2	4	3	3	2	2	1	



死锁预防 (deadlock prevention)

- **死锁预防 (deadlock prevention)**：破坏死锁的四个必要条件，使其至少一个不成立。
- 死锁的四个必要条件的破坏方法：
 - 互斥 (Mutual exclusion)：一些资源可以通过SP00Ling方式转化为非临界资源，如打印机、邮件队列
 - 持有并等待 (Hold and wait)：进程启动时一次性分配所需的所有资源，或者分配新资源前释放所有资源
 - 不可抢占 (No preemption)：直接抢占，对一些资源可能造成不可预期的后果
 - 循环等待 (Circular wait)：为每个资源编号，一个进程只能按编号从小到大申请资源
- 结论：除个别设备可以通过SP00Ling转化为可同时使用的非临界资源外，其他破坏方法基本上不具可操作性。



SPOOLing技术

- **SPOOLing技术：外设联机操作（Simultaneous Peripheral Operation On-Line），又称为假脱机输入/输出操作。**
- SPOOLing 技术的典型实例：将一台独占使用的打印机改造为可供多个用户同时使用的打印机。打印机是典型的需要独占使用的设备，但是用户在打印时，通常并不需要等待，仅需提交打印任务即可。
- 具体做法是：系统在磁盘开辟一个空间叫输出井。系统对于用户的打印输出，并不真正把打印机分配给该用户进程。而是先在输出井中申请一个空闲盘块区，并将要打印的数据送入其中。然后为用户申请并填写请求打印表，将该表挂到请求打印队列上。若打印机空闲，输出程序从请求打印队首取表，将要打印的数据从输出井传送到内存缓冲区，再进行打印，直到打印队列为空。
- 类似的例子还有：邮件发送队列等。



课后阅读

- 《操作系统概念 OSC（第9版）》第7章。
- 《现代操作系统 MOS（第4版）》第6章。
- 《深入理解计算机系统 CSAPP（第3版）》第12章。